Paluwagan

On-chain Rotating Savings & Credit Association (ROSCA) on the Bitcoin Cash **Chipnet** testnet

Project Report — Full Project Documentation

Generated August 2026

1. Overview

Paluwagan digitises the traditional Filipino *paluwagan* — a rotating savings and credit association — as a web application backed by smart contracts on the Bitcoin Cash Chipnet testnet. A group of members commits to pay a fixed contribution each round; every round, one member receives the entire pot. Contributions, rounds, and payouts are enforced on-chain by a CashScript contract (`RoundPot`) and orchestrated from a Laravel backend.

The application provides a **manager dashboard** (batch list, total value locked, per-batch round progress), a **members workspace** (contribution tracking, payout order, round controls), and a live **transaction history** pulled from the Chipnet block explorer showing every member contribution paid into the pot. Trust is replaced by verifiable on-chain records.

2. Key Features

- **ROSCA circles** — create batches with schedule (monthly / weekly / daily), contribution amount, and rotation policy.
- **Payout rotation** — *fixed* pays by membership position; *random* shuffles the payout order once per cycle so every member gets exactly one pot.
- **On-chain enforcement** — CashScript `RoundPot` contract locks contributions, lets the round recipient claim within a block window, and lets the organizer reclaim via `timeout()` if unclaimed.
- **Member tracking** — wallet address per member (name optional), contribution history, savings balance, status (Active / Released / Slashed), and auto-pay flag.
- **Contributions log** — live transaction history for each pot, sourced from `chipnet.bchexplorer.info`, showing date, wallet address, member name, amount, and txid.
- **Manager overview** — total value locked across all batches, contribution progress, and round status per batch.

- **Full auth stack** — Fortify-backed registration, login, email verification, two-factor authentication, and passkeys.

3. Technology Stack

Layer	Technology	Role
Backend	Laravel 13 (PHP 8.5)	API, domain logic, on-chain orchestration
Frontend	Inertia.js v3 + React 19 + Tailwind CSS 4	Server-driven SPA without API boilerplate
Routing	Laravel Wayfinder	Type-safe route helpers generated for the frontend
Auth	Laravel Fortify	Auth backend (2FA, passkeys, verification)
Smart contracts	CashScript (sBTC)	Per-round pot contract + Node worker simulator
Block explorer	Chipnet (bchexplorer.info)	Live on-chain transaction data (Fulcrum API)
Testing	Pest 5 / PHPUnit	Feature & unit coverage

> **Note:** the `contracts/` package (TypeScript) is compiled with CashScript and executed by `RoundService` via a Node worker process (`worker.ts`), which simulates and returns the contract address, payout txid, and phase for each round.

4. Architecture

A classic Laravel + Inertia monolith. Server-side controllers render Inertia pages; the React frontend is a client-side SPA driven by server data. On-chain state lives in the `contracts/` package; the Laravel side records round results in the ledger and pulls live contribution data from the Chipnet explorer API.

Component	Path	Responsibility

Web routes	`routes/web.php`	Batches, members, dashboard, settings
Controllers	`app/Http/Controllers`	Batch & member HTTP handlers
Services	`app/Services`	Round orchestration, explorer integration
Models	`app/Models`	Batch, Member, BatchMember, BatchContribution
Console	`app/Console/Commands`	`round:advance`, `round:expire`
Contracts	`contracts/src`	RoundPot CashScript + worker
Pages	`resources/js/pages`	Inertia+React views

5. Domain Model

Schema

Table	Key columns	Purpose
`users`	email, password, two-factor, passkeys	Organizer accounts (Fortify)
`members`	name *(nullable)*, wallet	Participants and their chipnet addresses
`batches`	name, status, schedule, rotation, contribution_sats, rounds_total, rounds_current, contract_address, pot_address, last_payout_tx	A savings circle
`batch_members`	batch_id, member_id, position, payout_order, status, auto_pay	Membership pivot; unique per batch+member
`batch_contributions`	batch_member_id, round, amount_sats, tx_id	Per-round ledger; unique per member+round

Relationships

- `Batch` ↔ `BatchMember` (hasMany) ↔ `Member` (belongsToMany via pivot).
- `BatchMember` → `BatchContribution` (hasMany); `Batch` aggregates contributions through `hasManyThrough`.

- Deletes cascade from `batch_members` down to `batch_contributions`; members survive batch deletion.

State Machines

Entity	States	Transitions
`Batch`	Forming → Active → Completed	created → Active on first round; Completed at rounds_current == rounds_total
`BatchMember`	Active / Released / Slashed	recipient Released after claiming pot

6. On-Chain Round Flow

RoundPot contract

`contracts/src/round_pot.cash` locks the round's pot (`contributionSats × memberCount`) into a P2SH contract between `startBlock` and `deadline`:

- `claim(pk, s)` — the round recipient withdraws the full pot (minus 2000 sats fee) while the round is live.
- `timeout(pk, s)` — after the deadline, the organizer reclaims the unclaimed pot, so funds are never stuck.

Round lifecycle

Step	Action	On-chain / ledger effect
1	Create batch + members	Rows in `batches`, `members`, `batch_members`
2	Assign payout order	Fixed = position; random = one shuffle per cycle
3	Advance round	worker returns contractAddress/txid; `BatchContribution` rows written for every member
4	Recipient claims	Member marked Released; `last_payout_tx` recorded

5	Round expires (optional)	Organizer reclaims pot via `timeout()`
6	Contribution log	`ChipnetExplorerService` pulls pot txs from the Chipnet API

Explorer integration

`ChipnetExplorerService` queries `https://chipnet.bchexplorer.info/api/bch/address/{pot}/txs` (cursor paginated, 100/page, up to 3 pages), keeps only transactions whose `vin` sender is a batch member's wallet, maps senders to member names (fallback `Member {position}`), formats amount and date, and caches the result for 60 seconds. Network failure degrades gracefully to an empty log.

7. HTTP API / Routes

Method	URI	Name	Handler
GET	`/`	home	Inertia page
GET	`/dashboard`	dashboard	Inertia page (auth+verified)
GET	`/batches`	batches.index	BatchController@index
POST	`/batches`	batches.store	BatchController@store
GET	`/batches/{batch}`	batches.show	BatchController@show
POST	`/batches/{batch}/advance`	batches.advance	BatchController@advance
POST	`/batches/{batch}/expire`	batches.expire	BatchController@expire
POST	`/members`	members.store	MemberController@store

Plus Fortify auth routes (login, register, 2FA, passkeys, verification) and settings routes (profile, security) in `routes/settings.php`.

8. Services & Commands

Class	Location	Responsibility

`RoundService`	`app/Services`	advance()/expire() — picks recipient, runs worker, records contributions
`ChipnetExplorerService`	`app/Services`	resolves pot transactions into the contributions log
`RoundAdvance`	`app/Console/Commands`	`php artisan round:advance {batch}`
`RoundExpire`	`app/Console/Commands`	`php artisan round:expire {batch}`

9. Frontend Pages

Page	Route	Contents
Home / Landing	`/`	Product landing page
Dashboard	`/dashboard`	Authenticated home
Batches	`/batches`	Batch cards with pot, round progress, status; total value locked card; create-batch dialog; wallet selector
Members	`/batches/{batch}`	Total balance, on-chain transaction history (Date / Wallet / Name / Txid / Amount), member cards with savings & progress, round controls (advance / expire)
Settings — Profile	`/settings/profile`	Profile, password, session management
Settings — Security	`/settings/security`	2FA, passkeys, account deletion
Auth	`/login`, `/register`, 2FA challenge, password reset, email verification	Fortify-backed flows

Manager UI highlights

- **Total Value Locked** — summed from all batches' on-ledger contributions (sats), rendered server-side.
- **Transaction History** — real pot transactions from the Chipnet explorer: date, wallet address, member name (or `Member N`), txid, amount.
- **Round Controls** — advance or expire the current round with server-side validation and flash feedback.
- **Member Cards** — address copy-to-clipboard, savings balance, contribution progress bar, due/remaining amounts, auto-pay toggle, status badge.

10. Testing

Pest 5 feature suite (“`php artisan test`”) covers batches, members, round orchestration, explorer integration, dashboard, and the Fortify auth flows. The on-chain explorer service is tested with faked HTTP responses (member mapping, nameless fallback, unreachable API, missing pot address).

Suite	Coverage
`BatchMemberControllerTest`	batch listing w/ derived pot & progress, members page, total value locked, member creation
`ChipnetExplorerServiceTest`	contribution log resolution, fallback labels, failure handling
`RoundServiceTest`	round advance / expire orchestration
`BatchRoscTest`	models, ledger, constraints, cascades
`CreateBatchTest` / `DashboardTest`	batch creation, dashboard
`tests/Feature/Auth`, `tests/Feature/Settings`	Fortify + settings flows

Tooling: Laravel Pint (style), Larastan (static analysis), ESLint/Prettier, TypeScript checks. CI script: `composer ci:check`.

11. Project Structure

Directory	Contents
`app/Models`	Batch, BatchMember, BatchContribution, Member, User
`app/Services`	RoundService, ChipnetExplorerService
`app/Http/Controllers`	BatchController, MemberController, Settings/*
`app/Console/Commands`	RoundAdvance, RoundExpire
`contracts/src`	round_pot.cash, worker.ts, keys.ts, demo.ts, artifacts
`database/migrations`	users, members, batches, batch_members, batch_contributions, Fortify columns

`resources/js/pages`	Inertia+React pages (manager, auth, settings, home)
`tests`	Pest feature/unit suites

12. Setup & Run

```
composer install
cp .env.example .env && php artisan key:generate
php artisan migrate --force
npm install && npm run build
```

Contracts: `cd contracts && npm install` (CashScript compilation via `cashc`).

Dev: `composer run dev` (or `php artisan serve` + `npm run dev`).

Single command alternative: `composer setup`.

13. Verification & Quality

- `vendor/bin/pint` — PHP code style.
- `php artisan test` — Pest feature/unit suites.
- `composer ci:check` — lint + static analysis + types + tests in one pass.
- Contracts verified with CashScript tests (`contracts/test`).

14. Future Work

- Wire real wallet funding so contributions are actually broadcast to Chipnet (currently simulated by the worker).
- Push notifications / scheduling for round deadlines (via `schedule`).
- Per-member contribution CSV export and receipts.
- Multi-organizer workspaces and role-based access.
- Deploy to the main Bitcoin Cash network after Chipnet validation.